

Leveraging Modern GPUs in Rust for the Andromeda SGBDR Engine (2018-2026)

Background and Objectives

The **Andromeda** project aims to develop a next-generation relational database system (SGBDR) for 2026 that offloads compute-intensive tasks—such as bench-marking, metric calculations, statistics, and plan-cache re-evaluation—to graphics processing units (GPUs). Unlike many GPU-driven projects, Andromeda does **not** focus on AI; its goals are to accelerate analytic workloads through parallel computation and high memory bandwidth. This document synthesizes research about modern GPU hardware from **NVIDIA** and **AMD** (2018–2026) and examines how the Rust ecosystem can leverage these GPUs. The aim is to provide actionable recommendations for designing Andromeda’s engine using Microsoft-style technical documentation.

GPU Hardware Landscape (2018–2026)

Recent GPUs offer different balances of precision, memory and performance. The table below highlights relevant architectures for data-analytics rather than AI inference.

NVIDIA GPU Architectures

Generation (year)	Key features relevant to analytics (non-AI)	Suitability
Turing – Volta successor (2018)	The Turing TU102 die introduced tensor cores and ray-tracing (RT) cores. It included 144 FP64 units (two per SM) but the FP64 throughput is 1/32 of FP32 , meaning double-precision workloads run much slower than single-precision 【308090920028930†L706-L714】. Memory is GDDR6, with up to ~616 GB/s bandwidth 【308090920028930†L859-L867】.	Adequate for basic parallel aggregation, but limited FP64 performance restricts heavy scientific calculations.
Ampere (A100/A40 – 2020)	Ampere introduced 3rd-generation tensor cores with support for FP64 matrix operations and structured sparsity. The A100 provides up to 40/80 GB of	Good for mixed-precision workloads; strong FP64 throughput makes it suitable for analytics and HPC.

Generation (year)	Key features relevant to analytics (non-AI)	Suitability
Hopper (H100/H200 – 2022/2024)	HBM2E memory and ~ 2 TB/s bandwidth. NVLink 3.0 provides 600 GB/s interconnect bandwidth 【947022055884292†L395-L423】 . Hopper’s 4th-generation tensor cores support FP8 and the Transformer Engine; it also increases on-chip L2 cache to ~80 MB and introduces distributed shared memory (DSMEM) for cross-SM synchronization 【947022055884292†L349-L357】 . H100 provides 80GB HBM3 with ~3.3TB/s bandwidth; the H200 variant offers 141 GB HBM3e and 4.8 TB/s bandwidth 【907921961041862†L83-L85】 . Hopper maintains strong FP64 performance and is optimized for HPC and analytics 【907921961041862†L40-L44】 .	Excellent for analytic workloads requiring double precision and high memory bandwidth.
Blackwell (B100/B200 – 2024–2026)	Blackwell departs from monolithic dies with a two-die chiplet design connected via a 10 TB/s inter-die link 【947022055884292†L360-L371】 . It packs 208 billion transistors, more than 2.5× Hopper’s ~80 billion 【947022055884292†L360-L371】 . According to patent analysis, Blackwell delivers 1.5× more AI compute FLOPS and 2× attention-layer acceleration versus Hopper while offering 25× better	Ideal for AI inference and large models; for data analytics requiring double precision, Hopper may still be preferable until future “Rubin” GPUs address FP64 performance.

Generation (year)	Key features relevant to analytics (non-AI)	Suitability
	energy efficiency per operation 【947022055884292†L349-L364】 . However, micro-architectural analyses show that Blackwell sacrifices some FP64 throughput in favor of low-precision (FP4/FP6/FP8) tensor operations 【907921961041862†L39-L55】 . These GPUs support up to 288 GB of HBM3e memory and ~8 TB/s bandwidth in NVL72 systems 【907921961041862†L90-L97】 .	

AMD GPU Architectures

Generation (year)	Key features relevant to analytics	Suitability
MI100 (CDNA 1 – 2020)	MI100 offered 32 GB HBM2 memory at ~1.2 TB/s bandwidth 【932009837730271†L123-L133】 and strong FP64 performance (~11.5 TFLOPs). ROCm open-source software provides a CUDA-like programming model.	Good entry point for GPU acceleration in Rust via ROCm/OpenCL.
MI250X (CDNA 2 – 2021)	Dual-chip design with 128 GB HBM2e (3.2 TB/s bandwidth). Supports strong FP64 throughput (~47.9 TFLOPs). ROCm matured with better library support 【932009837730271†L159-L170】 .	Suitable for HPC and analytics; limited by early ROCm maturity.
MI300X / MI325X (CDNA 3 – 2024)	MI300X provides 192 GB HBM3 and 5.3 TB/s bandwidth	Excellent for large in-memory analytics where memory capacity is critical.

Generation (year)	Key features relevant to analytics	Suitability
MI350X / MI355X (CDNA 4 – 2025)	【932009837730271†L176-L188】 . MI325X expands to 256 GB HBM3e with 6 TB/s bandwidth 【932009837730271†L159-L170】 . FP64 performance remains strong (55+ TFLOPs).	Provides huge memory and bandwidth; strong FP64 capabilities make it attractive for analytics; requires ROCm 7 for best performance.
	Built on TSMC 3 nm chiplets (10 chiplets, 185 billion transistors) 【263636406027050†L133-L143】 . Each GPU offers 256 compute units (16 384 cores) and supports up to 288 GB HBM3e memory with 8 TB/s bandwidth 【598164411054977†L524-L553】 . FP64 performance is 72–78.6 TFLOPs 【598164411054977†L544-L556】 【263636406027050†L190-L202】 . Rack-scale configurations (128 GPUs) reach 1.3 exaFLOPs of MXP4/6 performance 【598164411054977†L538-L542】 . The MI350 series supports new FP4 and FP6 formats for AI but still doubles as a high-precision engine.	
MI400 (“Helios” – 2026 preview)	Announced for 2026 with up to 432 GB HBM4 memory and 19.6 TB/s bandwidth; promises 20 PFLOPs FP8 and 40 PFLOPs FP4 performance 【598164411054977†L619-L627】 .	Future-proof for large-scale analytics and AI; availability expected in late 2026.

Implications for Andromeda

1. **Precision Requirements:** Data-analytics tasks such as variance, regression or heavy matrix multiplication often require double-precision (FP64) arithmetic. Turing GPUs

provide only 1/32 FP32 throughput for FP64 instructions [308090920028930†L706-L714] , while modern architectures like Hopper and MI350 deliver tens of TFLOPs FP64 performance [598164411054977†L544-L556] [598164411054977†L544-L556] . Choosing GPUs with strong FP64 capability is essential.

2. **Memory Capacity and Bandwidth:** Large in-memory datasets benefit from high bandwidth. MI350X/M355X and Blackwell NVL72 systems provide **8 TB/s** or more memory bandwidth [598164411054977†L524-L553] [947022055884292†L360-L371] ; Hopper H200 offers **4.8 TB/s** [907921961041862†L83-L85] . More memory reduces data movement between CPU and GPU and enables zero-copy techniques.
3. **Energy Efficiency:** For background tasks running continuously (e.g., statistics recalculation), energy efficiency matters. Blackwell claims **25× better energy efficiency per operation** than Hopper [947022055884292†L349-L364] , while AMD emphasises “tokens per watt” metrics in MI350 series marketing [263636406027050†L93-L100] . Selecting energy-efficient GPUs reduces operational costs.

Rust GPU Ecosystem (2026)

Rust’s safety guarantees and zero-cost abstractions make it attractive for high-performance systems. However, GPU programming in Rust remains nascent compared with CUDA C++. The following frameworks enable GPU compute in Rust by targeting different back-ends:

Framework / crate	Backend & scope	Strengths	Limitations
wgpu (WebGPU implementation)	Cross-platform API covering Vulkan, Metal, DirectX 12/11, OpenGL ES and WebGPU . Developed by the Rust community and funded by Google [926918539262445†L71-L108] .	Unified API; safe and portable; works on Windows, Linux, macOS (MoltenVK), Android and browsers. Suitable for compute workloads like scientific simulations and analytics [926918539262445†L114-L137] .	Lacks vendor-specific optimizations; performance can lag CUDA on Nvidia hardware [926918539262445†L148-L165] .
rust-gpu	Compiler project that turns regular Rust code into SPIR-V for Vulkan devices. Used by wgpu for shader code.	Allows writing GPU kernels in idiomatic Rust; integrates with naga for cross-target translation [730151876503216†L16-L38] [730151876503216†L55-L76] .	Limited support for complex language features; still evolving; currently targets Vulkan/SPIR-V only.
krnl	Safe, portable	High-level API;	Requires Vulkan 1.2

Framework / crate	Backend & scope	Strengths	Limitations
	compute kernels written inline in Rust; targets Vulkan 1.2+ with MoltenVK support on macOS. Provides debug backtraces and safe host-side access to buffers as slices or Vec 【677746990577422† L110-L121】 .	cross-platform; can intermix CPU and GPU code elegantly.	support; no vendor-specific acceleration; smaller ecosystem.
rustacuda / cuda_std	rustacuda is a safe wrapper over the CUDA Driver API and manages device memory and kernel launches 【108663057594965† L71-L105】 . cuda_std compiles Rust kernels into NVVM PTX for Nvidia GPUs (supported by Rust-GPU project).	Provides direct access to CUDA; can leverage libraries like cuBLAS, cuDNN through cudarc. Suitable for high-performance kernels on Nvidia hardware.	Nvidia-only; programming model is low-level; requires nightly Rust; limited multi-GPU and async support 【108663057594965† L71-L105】 .
cudarc	High-level crate that wraps CUDA libraries (driver, cuBLAS, cuDNN, cuSPARSE, cuSOLVER) and exposes abstractions like CudaContext, CudaStream, and CudaSlice<T> 【770891016288461† L103-L130】 .	Eases memory management, stream synchronization, and access to math libraries. Supports dynamic loading of CUDA libraries and multiple CUDA versions 【770891016288461† L160-L176】 .	Nvidia-only; depends on CUDA toolkit installation; still evolving.
ocl / rust_gpu_tools	ocl exposes the OpenCL API, while rust_gpu_tools provides a unified interface for OpenCL and CUDA kernels 【887321238635541†	Cross-vendor support; can target AMD GPUs via ROCm's OpenCL implementation.	OpenCL has lower maturity and tooling; performance may lag CUDA; debugging is more difficult.

Framework / crate	Backend & scope	Strengths	Limitations
vulkano	Safe wrapper over Vulkan; used primarily for graphics but includes compute pipelines.	Mature API; good for cross-platform compute; integrates with the Rust ecosystem.	More verbose than wgpu; requires manual synchronization and resource management.

Design Considerations for Andromeda's GPU Engine

1. Match Hardware to Workload

1. **Choose GPUs with strong FP64 performance:** For analytics and statistical workloads that require accurate double-precision arithmetic (variance, covariance, regression), select GPUs such as **NVIDIA Hopper** or **AMD MI350X/M355X**. Blackwell targets low-precision AI and may underperform on FP64 kernels [\[907921961041862†L39-L55\]](#) .
2. **Balance memory capacity and bandwidth:** Data-intensive queries benefit from large HBM memory and high bandwidth. MI350 series and Blackwell NVL72 systems offer **288 GB–8 TB/s** memory per GPU [\[598164411054977†L524-L553\]](#) [\[947022055884292†L360-L371\]](#) , while Hopper provides 80–141GB at 3.3–4.8 TB/s [\[907921961041862†L83-L85\]](#) . Avoid GPUs with only GDDR6 memory (e.g., gaming-oriented RTX cards) for heavy analytics.
3. **Consider energy efficiency:** Running continuous background tasks (recomputing statistics or plan weights) can be costly. Blackwell's chiplet design yields **25× energy efficiency** improvements [\[947022055884292†L349-L364\]](#) ; AMD stresses “tokens per watt” metrics [\[263636406027050†L93-L100\]](#) . Evaluate total cost of ownership when selecting hardware.

2. Architecting GPU-Backed Rust Modules

1. **Isolation of GPU kernels:** Separate compute-intensive kernels from business logic. Use crates like `cuda_std` or `rust-gpu` to write kernels in Rust, compile them to PTX or SPIR-V, and embed them in host code. For example, `krnl` allows writing kernels inline and retrieving host buffers as slices [\[677746990577422†L110-L121\]](#) .
2. **Unified memory management:** Use `cuda::CudaSlice<T>` or `wgpu's` buffer mapping to allocate device memory and ensure zero-copy transfers where possible. When offloading aggregated computations (e.g., sum, average, percentile), keep data in columnar layout (similar to Apache Arrow) to maximize memory throughput.
3. **Asynchronous execution and streams:** Overlap data transfers with computation using CUDA streams (`CudaStream`) [\[770891016288461†L160-L176\]](#) or `wgpu` command encoders. This is crucial when running background recomputation tasks alongside foreground query execution.
4. **Cross-platform fallback:** Provide a CPU fallback for systems without supported GPUs. `wgpu` and `rust_gpu_tools` can automatically run kernels on the CPU or via OpenCL.

This ensures that Andromeda remains portable across environments and avoids hard dependencies on specific hardware.

5. **Error handling and debugability:** GPUs can fail silently. Frameworks like `krnl` provide debug backtraces on kernel panics [【677746990577422†L110-L121】](#). Use such features to trace errors back to Rust source lines.

3. Integrating with the SGBDR Engine

1. **Vectorized execution and columnar storage:** Convert internal query operators (scan, join, aggregate) to operate on columns so they can be offloaded efficiently to GPU kernels. Align with the Apache Arrow memory model (used by DataFusion) to facilitate zero-copy transfers. Although DataFusion currently lacks stable GPU support [【660478220800553†L41-L61】](#), its columnar design is a good template.
2. **Plan cache recalibration:** Use the GPU to recompute statistics (histograms, correlation matrices, cost models) in background threads. Because GPUs excel at parallel reductions, tasks like recomputing cardinality estimates or cost weights can run asynchronously without blocking query execution. Employ `cudarc`'s BLAS libraries or custom kernels for matrix operations.
3. **Benchmark engine:** Integrate a benchmark harness that can replay workloads and measure latency with and without GPU acceleration. Use this to adjust heuristics (when to offload a query to GPU) and to validate performance on different hardware models.

4. Cross-Vendor Programming Strategy

1. **Primary back-end:** Use `wgpu` for cross-vendor compute pipelines. It abstracts over Vulkan, Metal and DirectX, enabling Andromeda to run on Nvidia, AMD and Intel GPUs—even within browsers or cloud environments [【926918539262445†L71-L108】](#).
2. **Vendor-specific optimization:** When targeting Nvidia hardware for heavy analytics, integrate `cudarc` and `cuda_std` to access optimized BLAS routines and advanced memory features. For AMD hardware, explore **ROCm 7** via OpenCL or HIP; crates like `ocl` or the emerging `hip-rs` (community project) can be used, though the tooling is less mature.
3. **Kernel compilation pipeline:**
 - Use `rust-gpu + naga` for SPIR-V kernels that run across Vulkan/Metal back-ends [【730151876503216†L55-L76】](#).
 - Use `cuda_std` for Nvidia PTX kernels compiled from Rust.
 - Consider writing small C/C++ kernels and calling them via FFI when Rust compiler support is lacking; `cudarc` can launch any PTX kernel and `rust_gpu_tools` abstracts over multiple back-ends.

Recommendations for 2026

1. **Hardware Selection:** Deploy the Andromeda engine on GPUs with high FP64 performance and sufficient memory. **Nvidia Hopper (H200)** and **AMD MI350X/M355X** are excellent choices, offering 72–79 TFLOPs FP64 performance and 141–288 GB memory [【598164411054977†L544-L556】](#) [【598164411054977†L544-L556】](#).

Use **Blackwell** or **MI400** for AI-heavy scenarios or when energy efficiency outweighs FP64 needs.

2. **Software Stack:** Base the compute back-end on **wgpu** for portability. Provide optional modules using **cudarc/cuda_std** for Nvidia-specific acceleration and **OpenCL** (via ocl) for AMD GPUs. Maintain a CPU fallback.
3. **Data Layout:** Adopt column-oriented storage and align memory to 64-byte cache lines. Use GPU-accelerated primitives (prefix sums, histograms) to compute statistics in the background. Keep intermediate data on GPU to avoid host round-trips.
4. **Concurrency Model:** Isolate GPU tasks in separate threads or async tasks. Use asynchronous command queues (CudaStream or wgpu command buffer) to schedule background recomputation concurrently with query processing.
5. **Monitoring and Profiling:** Instrument GPU kernels with timers and use vendor profiling tools (Nvidia NSight, AMD ROCm profiler) to identify bottlenecks. Regularly recalibrate heuristics based on actual workload characteristics.
6. **Future-proofing:** Track emerging frameworks like **hip-rs** (HIP bindings for AMD) and improvements in **rust-gpu**. Monitor Nvidia's **Rubin** architecture and AMD's **MI400** (Helios) for improved FP64 performance and memory capacity **【598164411054977†L619-L627】** . Update the engine to exploit these advancements as they become available.

Conclusion

The 2018–2026 period has seen tremendous advances in GPU architectures. For data-analytics workloads, double-precision performance, memory bandwidth and energy efficiency are more important than raw AI throughput. **Nvidia Hopper** and **AMD MI350** GPUs deliver the balance of precision and capacity needed for the Andromeda project. By coupling these modern GPUs with **Rust** frameworks like **wgpu**, **krnl**, and **cudarc**, Andromeda can offload heavy statistics and benchmarking tasks, recalibrate plan caches in the background, and provide a modern, efficient SGBDR engine ready for 2026. A thoughtful cross-vendor strategy and careful kernel design will ensure both portability and performance.